

Experiment No: 8

Make Use of any of the user authentication tool like Kerberos, NTLM, LDAP, RADIUS

Name: Rutuja Dundesh Mane
Div: A

Roll No: A44
PRN: 2324000718

Keycloak

ExamServerGUI.java:

```
import javax.swing.*.*;
import java.awt.*.*;
import java.io.*;
import java.net.*;
import java.util.Base64;

public class ExamServerGUI {

    private JTextArea logArea;
    private JFrame frame;

    public static void main(String[] args) {
        new ExamServerGUI().createAndShowGUI();
    }

    public void createAndShowGUI() {

        frame = new JFrame("Exam Server ");
        frame.setLayout(new BorderLayout());

        logArea = new JTextArea();
        logArea.setEditable(false);
        logArea.setBackground(Color.WHITE);
        logArea.setForeground(new Color(50, 50, 50));

        JScrollPane scrollPane = new JScrollPane(logArea);
        scrollPane.setBorder(BorderFactory.createTitledBorder("Live Logs"));

        JButton startBtn = new JButton("Start Server");

        startBtn.addActionListener(e -> {
            startBtn.setEnabled(false);
            log("Starting server...");
            startServer();
        });
    }
}
```

```

frame.add(startBtn, BorderLayout.NORTH);
frame.add(scrollPane, BorderLayout.CENTER);

frame.setSize(600, 450);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);

log("Server ready.");
}

private void log(String msg) {
    SwingUtilities.invokeLater(() -> logArea.append("> " + msg + "\n"));
}

public void startServer() {

    new Thread(() -> {
        try (ServerSocket serverSocket = new ServerSocket(5000)) {

            log("Server running on port 5000");

            while (true) {

                try (Socket socket = serverSocket.accept();
                    BufferedReader in = new BufferedReader(
                        new InputStreamReader(socket.getInputStream()));
                    PrintWriter out = new PrintWriter(socket.getOutputStream(), true)) {

                    String token = in.readLine();

                    if (!validateToken(token)) {
                        log("Invalid token");
                        out.println("Access Denied");
                        out.println("END");
                        continue;
                    }

                    log("Token validated");

                    String request = in.readLine();

                    if ("REQUEST_QP".equals(request)) {

                        String className = in.readLine();
                        String courseCode = in.readLine();
                        String semester = in.readLine();

                        log("Class: " + className);
                        log("Course: " + courseCode);
                        log("Semester: " + semester);
                    }
                }
            }
        }
    }).start();
}

```

```

        out.println("=== QUESTION PAPER ===");
        out.println("Class: " + className);
        out.println("Course: " + courseCode);
        out.println("Semester: " + semester);
        out.println("-----");

        switch (courseCode.trim()) {

            case "101":
                out.println("Subject: Information Systems (IS)");
                out.println("1. Define Information System.");
                out.println("2. Explain SDLC.");
                out.println("3. ERP advantages.");
                break;

            case "102":
                out.println("Subject: Advanced DBMS (ADBMS)");
                out.println("1. Normalization explain.");
                out.println("2. Transaction management.");
                out.println("3. Indexing types.");
                break;

            default:
                out.println("No question paper for this course code.");
                break;
        }

        out.println("END");
    }

    } catch (Exception e) {
        log("Error: " + e.getMessage());
    }
}

    } catch (IOException e) {
        log("Server error: " + e.getMessage());
    }
}).start();
}

public boolean validateToken(String token) {
    try {
        if (token == null || !token.contains(".")) return false;

        String[] parts = token.split("\\.");
        String payload = new String(Base64.getUrlDecoder().decode(parts[1]));

        return payload.contains("Student") || payload.contains("user");

    } catch (Exception e) {

```

```

        return false;
    }
}

```

ExamClientGUI.java:

```

import javax.swing.*;
import java.awt.*;
import java.io.*;
import java.net.*;
import java.net.http.*;
import java.net.URI;

public class ExamClientGUI {

    private JFrame frame;
    private JPanel mainPanel;
    private CardLayout cardLayout;

    private JTextArea qpArea;
    private JTextArea tokenDisplay;

    private String savedToken = null;

    static String TOKEN_URL =
        "http://localhost:8080/realms/OnlineExamRealm/protocol/openid-connect/token";
    static String CLIENT_ID = "exam-app";

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new ExamClientGUI().initGUI());
    }

    public void initGUI() {

        frame = new JFrame("Student Exam Portal");
        cardLayout = new CardLayout();
        mainPanel = new JPanel(cardLayout);

        // ===== LOGIN PANEL =====
        JPanel loginPanel = new JPanel(null);

        JLabel uLabel = new JLabel("Username:");
        uLabel.setBounds(50, 30, 100, 30);
        JTextField userField = new JTextField();
        userField.setBounds(160, 30, 160, 30);

        JLabel pLabel = new JLabel("Password:");
        pLabel.setBounds(50, 70, 100, 30);
        JPasswordField passField = new JPasswordField();
        passField.setBounds(160, 70, 160, 30);
    }
}

```

```

JButton tokenBtn = new JButton("Request Token");
tokenBtn.setBounds(160, 110, 160, 35);

JLabel tokenLabel = new JLabel("Token:");
tokenLabel.setBounds(50, 160, 100, 30);

tokenDisplay = new JTextArea();
tokenDisplay.setLineWrap(true);
tokenDisplay.setWrapStyleWord(true);
tokenDisplay.setEditable(false);

JScrollPane tokenScroll = new JScrollPane(tokenDisplay);
tokenScroll.setBounds(160, 160, 300, 60);

// ===== EXAM FIELDS =====
JLabel c1 = new JLabel("Class:");
c1.setBounds(50, 240, 100, 30);
JTextField classField = new JTextField();
classField.setBounds(160, 240, 160, 30);

JLabel c2 = new JLabel("Course Code:");
c2.setBounds(50, 280, 100, 30);
JTextField courseField = new JTextField();
courseField.setBounds(160, 280, 160, 30);

JLabel c3 = new JLabel("Semester:");
c3.setBounds(50, 320, 100, 30);
JTextField semField = new JTextField();
semField.setBounds(160, 320, 160, 30);

JButton qpBtn = new JButton("Request Question Paper");
qpBtn.setBounds(160, 360, 220, 35);
qpBtn.setEnabled(false);

loginPanel.add(uLabel); loginPanel.add(userField);
loginPanel.add(pLabel); loginPanel.add(passField);
loginPanel.add(tokenBtn);
loginPanel.add(tokenLabel);
loginPanel.add(tokenScroll);

loginPanel.add(c1); loginPanel.add(classField);
loginPanel.add(c2); loginPanel.add(courseField);
loginPanel.add(c3); loginPanel.add(semField);
loginPanel.add(qpBtn);

// ===== EXAM PANEL =====
JPanel examPanel = new JPanel(new BorderLayout());

qpArea = new JTextArea();
qpArea.setEditable(false);

```

```

examPanel.add(new JScrollPane(qpArea), BorderLayout.CENTER);

mainPanel.add(loginPanel, "LOGIN");
mainPanel.add(examPanel, "EXAM");

// ===== STEP 1: GET TOKEN =====
tokenBtn.addActionListener(e -> {

    String token = getToken(
        userField.getText(),
        new String(passField.getPassword())
    );

    if (token == null) {
        JOptionPane.showMessageDialog(frame, "Invalid login!");
        return;
    }

    savedToken = token;

    tokenDisplay.setText(token); // immediate display

    qpBtn.setEnabled(true);

    JOptionPane.showMessageDialog(frame, "Token received successfully!");
});

// ===== STEP 2: REQUEST QUESTION PAPER =====
qpBtn.addActionListener(e -> {

    String className = classField.getText();
    String courseCode = courseField.getText();
    String semester = semField.getText();

    cardLayout.show(mainPanel, "EXAM");

    fetchQuestionPaper(savedToken, className, courseCode, semester);
});

frame.add(mainPanel);
frame.setSize(700, 550);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
}

// ===== TOKEN REQUEST =====
public static String getToken(String username, String password) {
    try {
        HttpClient client = HttpClient.newHttpClient();

```

```

String body =
    "client_id=" + CLIENT_ID +
    "&grant_type=password" +
    "&username=" + username +
    "&password=" + password;

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create(TOKEN_URL))
    .header("Content-Type", "application/x-www-form-urlencoded")
    .POST(HttpRequest.BodyPublishers.ofString(body))
    .build();

HttpResponse<String> response =
    client.send(request, HttpResponse.BodyHandlers.ofString());

if (response.statusCode() == 200) {
    return response.body()
        .split("\"access_token\":\")[1]
        .split("\"")[0];
}

} catch (Exception e) {
    e.printStackTrace();
}
return null;
}

// ===== QUESTION PAPER REQUEST =====
private void fetchQuestionPaper(String token,
    String className,
    String courseCode,
    String semester) {

    new Thread(() -> {
        try (Socket socket = new Socket("localhost", 5000);
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()))) {

            out.println(token);
            out.println("REQUEST_QP");
            out.println(className);
            out.println(courseCode);
            out.println(semester);

            String line;
            StringBuilder sb = new StringBuilder();

            while ((line = in.readLine()) != null) {
                if (line.equals("END")) break;
                sb.append(line).append("\n");
            }
        }
    }).start();
}

```

```

    }

    SwingUtilities.invokeLater(() ->
        qpArea.setText(sb.toString())
    );

    } catch (Exception e) {
        SwingUtilities.invokeLater(() ->
            qpArea.setText("Server Error")
        );
    }
    }).start();
}
}

```

Output:

